

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Análisis de sistemas II

Análisis, Rediseño Arquitectónico y Contrato API

Propuesta de Modernización del Sistema Legado de Ventas e Integración Intermodular

Integrantes:

Anderson Cristofer Rodríguez Pivaral	9989-23-495
Pablo Jafet Morales Zacarías	9989-23-11401
Ricardo Emilio Guidos Pérez	9989-15-18163
Jaime Ricardo Garrido Giron	9989-23-21073

Guatemala 11 de septiembre de 2026

1. INTRODUCCIÓN

Se analiza un sistema de gestión empresarial monolítico heredado que centraliza las operaciones de tres departamentos clave: Ventas (núcleo transaccional), Recursos Humanos y Soporte Técnico. El propósito es realizar la ingeniería inversa del estado actual y proponer un rediseño arquitectónico enfocado en extraer y modernizar el Módulo de Ventas.

Se busca solucionar la saturación de la base de datos compartida y el alto nivel de acoplamiento que provoca que los picos transaccionales en ventas degraden el rendimiento general, afectando la operatividad de los otros departamentos y complicando el despliegue de nuevas funcionalidades.

Se propone migrar hacia una arquitectura de Microservicios orientada a APIs RESTful, aislando el módulo de Ventas con su propia base de datos, implementando un API Gateway y estableciendo contratos claros (OpenAPI) para asegurar escalabilidad, alta disponibilidad y despliegues independientes.

2. Contexto del Sistema Legado

Actualmente, la organización opera sobre un sistema monolítico heredado donde el núcleo transaccional de ventas, la atención técnica y la gestión de recursos humanos comparten el mismo código fuente base y almacenamiento físico. A continuación se caracterizan el lenguaje, la base de datos y los principales puntos de dolor identificados:

2.1 Descripción del sistema actual

- **Propósito del sistema:** Gestionar integralmente las ventas, comisiones de empleados y garantías de clientes.
- **Usuarios principales:** Vendedores, Cajeros, Agentes de Soporte Técnico y Analistas de RRHH.
- **Procesos de negocio involucrados:** Facturación y cobro (Ventas), cálculo de planillas/comisiones (RRHH) y validación de vigencia de garantías (Soporte Técnico).
- **Tipo de arquitectura actual:** Monolítica tradicional centralizada.
- **Principales funcionalidades:** Registro de pedidos, consulta de inventario, emisión de tickets de soporte y cálculo de nómina.

2.2 Tecnología y Base de Datos Actual

El sistema está desarrollado bajo una arquitectura monolítica tradicional monolingüe (PHP/Java Enterprise) conectada a una única base de datos relacional centralizada (MySQL/PostgreSQL).

2.3 Puntos de dolor identificados

- **Cuellos de Botella en Rendimiento:** Durante cierres de mes o períodos de alta demanda transaccional en Ventas, la carga masiva satura las conexiones de la base de datos compartida, ralentizando o bloqueando la consulta de garantías en Soporte Técnico y la generación de reportes en Recursos Humanos.
- **Alto Acoplamiento:** Los módulos operativos no están aislados. Cualquier cambio o falla en el módulo transaccional de ventas afecta de manera directa la disponibilidad operacional de Soporte y RRHH.
- **Despliegues Complejos y Riesgosos:** Ajustes menores en el cálculo de comisiones de RRHH o validación de garantías requieren compilar, probar y desplegar la aplicación completa, incrementando sustancialmente el riesgo de tiempo de inactividad (downtime).
- **Dificultad de Escalabilidad Vertical** La única manera de absorber picos de tráfico es escalando verticalmente el servidor central, lo que genera costos de infraestructura elevados e insostenibles a largo plazo.

3. INGENIERÍA INVERSA DEL SISTEMA ACTUAL

3.1 Análisis de la arquitectura existente: Actualmente, el sistema funciona de manera secuencial y síncrona. Cuando un usuario de cualquier departamento interactúa con la interfaz web, la petición HTTP llega a un único servidor de aplicaciones. Esta capa lógica monolítica procesa reglas de negocio mezcladas (ej. validando una venta y al mismo tiempo actualizando la comisión del vendedor) e impacta directamente a un gestor de base de datos relacional centralizado (MySQL), convirtiendo a la base de datos en un punto único de falla (SPOF) y en un embudo de procesamiento.

El siguiente diagrama de componentes UML refleja la arquitectura monolítica actual del sistema, evidenciando el acoplamiento directo entre los módulos operativos y la dependencia única hacia un almacenamiento relacional compartido.

[\[Abrir diagrama 1\]](#)

Archivo: [docs/legacy/arquitectura_legacy.png](#)

Fuente editable: [docs/legacy/arquitectura_legacy.drawio](#)

4. ANÁLISIS DE ALTERNATIVAS ARQUITECTÓNICAS

4.1 Alternativas consideradas

A continuación se presenta el cuadro evaluativo que compara el sistema monolítico heredado frente a alternativas de rediseño arquitectónico (Monolito Modular, Event-Driven Architecture y la propuesta basada en Microservicios / RESTful):

Criterio	Monolito Heredado (Actual)	Monolito Modular	Event-Driven Architecture (EDA)	Microservicios / RESTful (Propuesta)
Pros	Simplicidad inicial de desarrollo; baja complejidad de red interna.	Código mejor estructurado por dominios; bajo costo de infraestructura inicial.	Desacoplamiento asíncrono extremo; alta resiliencia ante picos de tráfico.	Desacoplamiento total; escalabilidad independiente por módulo; independencia tecnológica; despliegues continuos sin afectación global.
Contras	Alto acoplamiento; fallas en cascada; escalabilidad costosa e ineficiente.	Sigue compartiendo la misma base de datos y recursos del servidor físico.	Alta complejidad de depuración, trazabilidad y consistencia de datos eventual.	Mayor complejidad operacional inicial (requiere API Gateway, orquestación, monitoreo).
Riesgos	Colapso del sistema en picos de venta; imposibilidad de evolucionar.	Mantenimiento limitado si las fronteras de código se rompen con el tiempo.	Duplicidad de eventos; altas curvas de aprendizaje para el equipo de desarrollo.	Latencia de red intermodular si no se diseñan adecuadamente las interfaces de integración.

4.2 Matriz considerada

CRITERIO	MONOLITO	MICROSERVICIOS (VENTAS)	MONOLITO MODULAR	EVENT-DRIVEN
ESCALABILIDAD	Baja. Solo escala verticalmente.	Alta. Escala horizontalmente solo el módulo requerido.	Media. Escala todo el bloque de código, pero mejor organizado.	Muy Alta. Absorbe picos fácilmente mediante colas.
MANTENIBILIDAD	Baja. Alto acoplamiento.	Alta. Bases de código separadas y contratos API claros.	Media-Alta. Fronteras lógicas claras, pero misma base física.	Media. Alta complejidad para rastrear flujos.
DISPONIBILIDAD	Baja. Falla un módulo, falla todo.	Alta. Falla RRHH, las Ventas continúan.	Media. Un bug crítico de memoria afecta a todo.	Alta. Los mensajes se retienen si un servicio cae.
SEGURIDAD	Media. Un solo perímetro, pero si entra un atacante, tiene acceso total.	Alta. Seguridad por API Gateway y tokens JWT granulares.	Media. Igual al monolito tradicional.	Media. Requiere asegurar el bus de eventos.
COMPLEJIDAD	Baja. Fácil de entender inicialmente.	Alta. Requiere orquestación, CI/CD y monitoreo.	Media. Requiere disciplina en el código.	Muy Alta. Manejo de asincronía y consistencia eventual.
COSTO	Alto (Infra. Vertical)	Medio-Alto (Operacional y Nube)	Bajo-Medio	Alto (Infraestructura de colas)
RIESGOS	Colapso total del negocio.	Latencia de red si se comunican mal las APIs.	Las fronteras modulares se pueden romper fácilmente.	Duplicidad de eventos y desincronización de datos.

4.3 Arquitectura seleccionada

Arquitectura propuesta: Microservicios expuestos mediante API RESTful

4.3 Justificación técnica

La selección de un estilo arquitectónico basado en Microservicios y Servicios Web RESTful con un API Gateway se fundamenta en el análisis riguroso de los siguientes atributos de calidad:

Escalabilidad

El módulo de Ventas actúa como el motor transaccional central y experimenta la mayor variabilidad de carga. Al aislar este módulo en su propio microservicio, es posible escalar horizontalmente únicamente las instancias de Ventas ante picos de demanda (e.g. promociones o campañas), sin necesidad de malgastar recursos de cómputo en los módulos de Soporte Técnico o Recursos Humanos.

Mantenibilidad

La definición formal de contratos de interfaz mediante la especificación OpenAPI 3.0 establece un desacoplamiento estricto entre los equipos de desarrollo. El equipo de Ventas puede modificar la lógica interna, refactorizar el código o cambiar la base de datos subyacente de forma autónoma, siempre que se respeten los endpoints expuestos (tales como `/sales/rep/{vendedorId}` o `/sales/validate-warranty/{saleId}`), eliminando los riesgos de regresión en otros módulos.

Disponibilidad

La arquitectura distribuida previene los puntos únicos de falla. Mediante la implementación de un API Gateway provisto de políticas de limitación de tasa y aislamiento de fallas, se garantiza que consultas intensivas o pesadas realizadas desde RRHH para la generación de reportes de comisiones no degraden ni interrumpan el procesamiento transaccional de ventas ni los cobros con pasarelas externas.

Seguridad

Se centraliza y estandariza la seguridad mediante la emisión y validación de tokens JSON Web Tokens (JWT) firmados con algoritmos HMAC SHA-256. Adicionalmente, se aplica un esquema de Control de Acceso Basado en Roles (RBAC), asegurando que los usuarios del

módulo de Técnico de Soporte únicamente accedan a endpoints de validación de garantías y compras, impidiendo el acceso a endpoints de nómina y comisiones restringidos de RRHH. Toda la comunicación externa e intermodular se fuerza sobre canales cifrados TLS 1.3 (HTTPS).

5. CONTRATO API – OPENAPI 3.0

5.1 Descripción de la API

- **Nombre:** API RESTful del Módulo de Ventas
- **Propósito:** Exponer las capacidades transaccionales del módulo de ventas e integrar de forma segura e independiente los flujos con Recursos Humanos (cálculo de comisiones) y Soporte Técnico (validación de garantías).
- **Versión:** 1.0.0 (OpenAPI 3.0.3)
- **Recursos principales:** /auth, /payments, /sales
- **Consumidores:** Módulo de Recursos Humanos, Módulo de Soporte Técnico, Pasarelas de Pago externas y Clientes/Vendedores mediante aplicaciones web/móviles.

5.2 Endpoints

Método	Endpoint	Operación	Autenticación
POST	/auth/login	Autenticación de usuarios y emisión de token JWT	No (Público)
POST	/payments/process	Procesamiento transaccional de cobros de ventas	Sí (Bearer JWT)
GET	/sales/rep/{vendedorId}	Consulta de historial de ventas por vendedor (para RRHH)	Sí (Bearer JWT)
GET	/sales/validate-warranty/{saleId}	Validación de existencia de venta y vigencia de garantía (para Soporte)	Sí (Bearer JWT)
GET	/sales/customer/{clienteId}	Consulta de historial de compras de un cliente	Sí (Bearer JWT)
GET	/sales/summary	Resumen e indicadores consolidados de ventas por período	Sí (Bearer JWT)

5.3 Detalle de endpoints

- **POST /auth/login:** Recibe un objeto JSON con correo y contraseña (LoginRequest), valida las credenciales y devuelve el token JWT con su tiempo de expiración (LoginResponse). Retorna 200 OK o 401 Unauthorized.
- **POST /payments/process:** Recibe el monto, moneda y un token seguro de tarjeta (PaymentRequest) para procesar la transacción financiera. Retorna 201 Created con el ID de transacción (PaymentResponse) o 400/401/500 en caso de fallo.

- **GET /sales/rep/{vendedorId}**: Recibe el parámetro en ruta {vendedorId} y un rango de fechas optativo por consulta (startDate, endDate). Retorna un DTO SalesRepResponse con el total facturado y la lista de ventas. Retorna 200, 400, 401, 404 o 500.
- **GET /sales/validate-warranty/{saleId}**: Recibe el ID de venta {saleId} y retorna un DTO WarrantyValidationResponse indicando si la garantía sigue activa y los días restantes. Retorna 200, 401, 404 o 500.
- **GET /sales/customer/{clienteId}**: Recibe el ID o NIT/DPI del cliente {clienteId} y retorna un DTO CustomerSalesResponse con el acumulado de compras e historial. Retorna 200, 401, 404 o 500.
- **GET /sales/summary**: Recibe obligatoriamente startDate y endDate por query params y retorna un DTO SalesSummaryResponse con los totales facturados, ticket promedio y vendedores activos. Retorna 200, 400, 401 o 500.

5.4 DTOs

DTO	Tipo	Descripción
LoginRequest	Request Body	Contiene email (string) y password (string).
LoginResponse	Response (200)	Contiene token (JWT string), tokenType ("Bearer") y expiresIn (segundos).
PaymentRequest	Request Body	Contiene amount (float), currency (string) y cardToken (string).
PaymentResponse	Response (201)	Contiene transactionId (string), status ("APPROVED") y timestamp (date-time).
SalesRepResponse	Response (200)	Contiene vendedorId, totalFacturado, cantidadTransacciones y un arreglo de SaleItem.
WarrantyValidationResponse	Response (200)	Contiene saleId, clienteId, producto, fechaCompra, tieneGarantia (boolean) y diasRestantesGarantia (int).
CustomerSalesResponse	Response (200)	Contiene clienteId, totalCompras, montoAcumulado y un arreglo de SaleItem.
SalesSummaryResponse	Response (200)	Contiene periodoInicio, periodoFin, totalFacturadoGeneral, totalVentasRealizadas, ticketPromedio y vendedoresActivos.
ErrorResponse	Response (Error)	Objeto estandarizado de falla con code (string), message (string) y timestamp (date-time).

5.5 Códigos HTTP

- **200 OK**: Petición exitosa en lecturas y autenticación (GET / POST Login).
- **201 Created**: Recurso o transacción financiera creada exitosamente (POST Payments).
- **400 Bad Request**: Formato JSON inválido, datos obligatorios faltantes o rango de fechas incorrecto.
- **401 Unauthorized**: Token JWT omitido, alterado, expirado o credenciales de acceso erróneas.

- **404 Not Found:** El recurso solicitado (vendedor, cliente, venta o factura) no existe en la BD.
- **500 Internal Server Error:** Excepción no controlada en el servidor, falla de conexión con la BD o la pasarela.

5.6 Seguridad

Se implementa el esquema de seguridad Bearer Authentication (JWT) en la cabecera HTTP Authorization: Bearer <token_jwt>. Los tokens son firmados con el algoritmo HMAC SHA-256. Se exige control de acceso basado en roles (RBAC) para limitar que cada módulo consumidor interactúe únicamente con los endpoints permitidos para su función operativa.

Evidencias del contrato API

Vínculo al Archivo 1 – Especificación OpenAPI: [Abrir OpenAPI YAML]

Archivo: [docs/api/openapi.yaml](#)

Vínculo a la Evidencia 1 – Swagger UI: [Abrir Captura Swagger UI]

Archivo: [docs/api/swagger_ui.png](#)

6 DISEÑO DE ARQUITECTURA C4

6.1 Nivel 1 – Diagrama de contexto

El diagrama de contexto establece los límites del sistema rediseñado.

- **Usuarios:** Clientes (que consultan productos y realizan compras) y Administradores (que gestionan el inventario y las ventas).
- **Sistema principal:** El Sistema de Gestión de Ventas, el cual centraliza la lógica transaccional.
- **Sistemas externos:** Pasarela de Pagos externa (encargada de procesar el cobro financiero).
- **Relaciones:** Los usuarios interactúan directamente con el sistema de ventas, el cual delega exclusivamente el procesamiento de la tarjeta a la pasarela externa.

Vínculo al diagrama 2 -C4 Nivel 1: Contexto:

[docs/c4/c4_nivel1_contexto.png](#)

Vínculo al archivo fuente:

[docs/c4/src/c4_nivel1_contexto.drawio](#)

6.2 Nivel 2 – Diagrama de contenedores

El diagrama de contenedores desglosa el Sistema de Gestión de Ventas en sus unidades de despliegue independientes.

- **Web/Mobile:** Aplicación Web que sirve como interfaz gráfica para clientes y administradores.
- **API Backend:** Un contenedor basado en Spring Boot que expone la API RESTful detallada en el contrato OpenAPI. Se comunica vía HTTP/REST con el frontend.
- **Base de datos:** Contenedor de base de datos relacional (MySQL/PostgreSQL) dedicado exclusivamente al dominio de ventas.
- **Redis:** Contenedor de caché en memoria para almacenar información consultada frecuentemente (como el catálogo de productos), implementando el patrón Cache-Aside.
- **Servicios externos:** Pasarela de Pagos externa.

Vínculo al Diagrama 3 – C4 Nivel 2: Contenedores:

[docs/c4/c4_nivel2_contenedores.png](#)

Vínculo al archivo fuente:

[docs/c4/src/c4_nivel2_contenedores.drawio](#)

6.3 Nivel 3 – Diagrama de componentes

El diagrama de componentes hace zoom dentro del contenedor "API Backend" para mostrar su estructura interna y la separación de responsabilidades.

- **Controllers:** Punto de entrada de las peticiones (Controlador de Autenticación, Controlador de Ventas).
- **Middlewares:** Filtro de seguridad que intercepta las peticiones para validar la firma y vigencia del Token JWT.
- **DTOs:** Objetos de transferencia de datos (LoginRequest DTO, VentaDTO) que definen el contrato de entrada/salida.
- **Services:** Capa que orquesta la lógica de negocio pura (Servicio de Autenticación, Servicio de Ventas).
- **Repositories:** Capa de acceso a datos que abstrae las consultas SQL hacia la base de datos subyacente.

Vínculo al Diagrama 4 – C4 Nivel 3: Componentes:

[docs/c4/c4_nivel3_componentes.png](#)

Vínculo al archivo fuente:

[docs/c4/src/c4_nivel3_componentes.drawio](#)

7. DISEÑO DE PERSISTENCIA DE DATOS

7.1 Tecnología seleccionada

Motor de base de datos: MySQL (con motor InnoDB).

Tipo: Relacional (RDBMS).

Justificación de la selección: El módulo de ventas maneja transacciones financieras y cálculos exactos que alimentan a Recursos Humanos y Soporte. Un modelo relacional bajo el motor InnoDB de MySQL asegura propiedades ACID, garantizando la integridad referencial y previniendo inconsistencias (por ejemplo, evitar que una garantía quede huérfana si falla el registro de un pago). Además, la amplia experiencia del equipo de desarrollo con MySQL asegura una implementación ágil y libre de errores de configuración.

7.2 Modelo de datos

El modelo de datos relacional se estructura alrededor de la entidad pivote `ventas`. Esta entidad registra la cabecera transaccional y se vincula directamente con `clientes` (para mantener el historial de compras) y `usuarios` (específicamente vendedores, para el cálculo de comisiones por parte de Recursos Humanos).

Los ítems comercializados en cada transacción se gestionan de forma desacoplada en la tabla `venta_detalle`, la cual se relaciona con la entidad `productos` para registrar el catálogo, los precios y los períodos de garantía activa. Por su parte, la tabla `pagos` se vincula de manera directa a `ventas` (relación 1 a 1 o 1 a N según reintentos) para almacenar el historial de transacciones financieras procesadas exitosamente a través de la pasarela de pagos.

Vínculo al Diagrama 5 – Modelo de Datos:

Archivo: docs/database/modelo_datos.png

7.3 Entidades principales

ENTIDAD	RESPONSABILIDAD	ATRIBUTOS PRINCIPALES
USUARIOS	Almacenar las credenciales de acceso y roles del sistema (Vendedores, Administradores).	usuario_id (PK), email, password_hash, rol, estado
CLIENTES	Registrar la información general de los compradores.	cliente_id (PK), documento_identidad, nombre, correo, telefono
PRODUCTOS	Gestionar el catálogo comercial, precios de venta, stock disponible y meses de garantía legal.	producto_id (PK), sku, nombre, precio, meses_garantia, stock
VENTAS	Consolidar la cabecera de la transacción de venta, asociando al cliente y al vendedor responsable.	venta_id (PK), cliente_id (FK), vendedor_id (FK), fecha, total, estado
VENTA_DETALLES	Almacenar el detalle de los productos adquiridos en cada venta con sus cantidades y subtotales.	detalle_id (PK AUTO_INCREMENT), venta_id (FK), producto_id (FK), cantidad, precio_unitario, subtotal
PAGOS	Registrar la confirmación financiera y el estado de la transacción emitida por la pasarela externa.	pago_id (PK AUTO_INCREMENT), venta_id (FK), transaccion_pasarela, metodo_pago, monto, estado_pago, fecha_pago

8. SCRIPT DE BASE DE DATOS

El script ejecutable DDL (`schema.sql`) automatiza la creación completa de la estructura de la base de datos en MySQL y la carga de datos de prueba iniciales. El script contiene los siguientes componentes técnicos:

- **Creación de tablas:** Definición explícita de cada tabla utilizando el motor `ENGINE=InnoDB` y codificación de caracteres `utf8mb4` para soporte internacional.
- **Claves primarias:** Asignación de identificadores únicos (`PRIMARY KEY`), empleando identificadores en formato cadena (`VARCHAR`) para entidades de negocio principales (ventas, productos, usuarios) y autoincrementales (`INT AUTO_INCREMENT`) para transacciones internas.
- **Claves foráneas:** Definición de restricciones `FOREIGN KEY` con políticas de integridad referencial como `ON DELETE RESTRICT` (para impedir la eliminación accidental de clientes o vendedores con ventas asociadas) y `ON DELETE CASCADE` (para la limpieza automática de detalles de venta).
- **Restricciones:** Incorporación de restricciones `NOT NULL` en campos obligatorios, `UNIQUE` en correos, documentos y SKUs de productos, y enumeraciones (`ENUM`) para estandarizar roles y estados.

- **Índices:** Creación de índices secundarios B-Tree sobre las columnas `fecha` y `vendedor_id` de la tabla `ventas` para acelerar los tiempos de respuesta en las consultas de reportes consolidados (`/sales/summary` y `/sales/rep/{vendedorId}`).
- **Datos de prueba (seeds):** Inserción de registros iniciales (`INSERT INTO`) que permiten probar inmediatamente el funcionamiento de los 6 endpoints definidos en la API OpenAPI.

Vínculo al Archivo 3 – Script de Base de Datos:

<docs/database/schema.sql>

9. ESTRATEGIA DE CACHE

9.1 Justificación

Información almacenada en caché: Se almacenarán los datos de lectura intensiva y baja frecuencia de modificación, tales como el catálogo general de productos, los resúmenes de ventas mensuales solicitados por Recursos Humanos (`/sales/summary`) y el historial consolidado de compras por cliente.

Por qué se utilizará caché: La implementación de Redis como motor en memoria permite interceptar peticiones recurrentes de lectura sin necesidad de ejecutar consultas analíticas complejas en MySQL.

Problema a resolver: Elimina el cuello de botella de E/S (Entrada/Salida) en el servidor de base de datos MySQL, reduce la latencia de respuesta de la API de segundos a milisegundos y garantiza la alta disponibilidad del sistema ante picos de concurrencia.

9.2 Patrón seleccionado

Se seleccionó el patrón **Cache-Aside** (también conocido como *Lazy Loading*).

- **Funcionamiento:** Cuando llega una petición de lectura a la API backend, esta consulta primero en Redis usando una clave específica. Si los datos se encuentran en caché (*Cache Hit*), se retornan de inmediato al cliente. Si los datos no existen (*Cache Miss*), la API consulta a la base de datos MySQL, guarda el resultado en Redis fijando un tiempo de vida (TTL) y finalmente devuelve la respuesta.
- **Justificación:** Otorga un control explícito sobre qué información reside en memoria y evita saturar Redis con datos que no son consultados con frecuencia.

9.3 Caché Keys y TTL

Información	Cache Key	TTL (Time To Live)
Catálogo general de productos	<code>products:catalog:all</code>	1 hora (3600s)
Historial de ventas por cliente	<code>sales:customer:{clienteId}</code>	15 minutos (900s)
Historial de ventas por vendedor	<code>sales:rep:{vendedorId}</code>	15 minutos (900s)
Resumen consolidado mensual de ventas	<code>sales:summary:{YYYY-MM}</code>	1 hora (3600s)

9.4 Política de invalidación

Para asegurar la coherencia eventual entre la caché Redis y la base de datos MySQL, se aplica una política de invalidación activa ante mutaciones de datos:

- **Creación de información (POST):** Cuando se procesa con éxito un cobro en `/payments/process` o se registra una venta, la API elimina explícitamente las claves afectadas en Redis.
- **Actualización / Eliminación de información (PUT / DELETE):** Si se actualiza el stock o precio de un producto, la API invalida de inmediato la clave `products:catalog:all`.
- **Efecto:** La siguiente petición de lectura generará un *Cache Miss*, forzando a la API a consultar los datos frescos de MySQL y repoblar la caché en Redis.

9.5 Flujo de caché

El diagrama adjunto ilustra la interacción entre el Cliente (o módulo de RRHH), la API Backend, Redis y la Base de Datos MySQL, cubriendo los tres escenarios operativos: *Cache Hit*, *Cache Miss* y la *Invalidación activa* tras registrar una nueva venta.

Vínculo al Diagrama 6 – Estrategia de Caché:

docs/cache/estrategia_cache.png

10. MATRIZ DE TRAZABILIDAD

La matriz de trazabilidad permite demostrar la alineación y coherencia entre los requerimientos de negocio, los problemas identificados en el sistema legado, las soluciones de arquitectura diseñadas y sus respectivas evidencias en el repositorio.

Necesidad / Requisito	Solución	Evidencia
Problema de escalabilidad	Arquitectura propuesta en capas desacopladas.	[Ver Análisis de Arquitectura]
Acoplamiento del legado	Rediseño y delimitación del sistema de ventas.	[Vínculo al diagrama 1]
Exposición de funcionalidades	API RESTful orientada a recursos.	[Vínculo al OPENAPI]
Autenticación	Esquema de seguridad Bearer JWT (HMAC SHA-256).	[Vínculo al OPENAPI]
Arquitectura propuesta	Planos bajo el modelo C4 (Niveles 1, 2 y 3).	[Vínculo al diagrama 2] [Vínculo al diagrama 3] [Vínculo al diagrama 4]
Persistencia	Modelo de Datos Entidad-Relación (MySQL / InnoDB).	[Vínculo al diagrama 5]
Creación de BD	Script DDL ejecutable con restricciones y seeds.	[Vínculo al schema.sql]
Rendimiento	Estrategia de Caché en memoria con Redis (Cache-Aside).	[Vínculo al diagrama 6]

11. VALIDACIÓN DE LA SOLUCIÓN

11.1 Validación del contrato API

La validez técnica de la especificación OpenAPI 3.0.3 se comprobó mediante las siguientes pruebas:

- **Validez del OpenAPI:** La sintaxis YAML fue validada en *Swagger Editor*, confirmando cero errores de sintaxis y cumplimiento estricto del estándar OpenAPI 3.0.3.
- **Visualización en Swagger UI:** Se generó la documentación interactiva en Swagger UI, permitiendo inspeccionar de forma gráfica las rutas, parámetros, esquemas y la funcionalidad *Try it out*.
- **Endpoints:** Se verificó la correcta exposición de los 6 endpoints requeridos: `/auth/login`, `/payments/process`, `/sales/rep/{vendedorId}`, `/sales/validate-warranty/{saleId}`, `/sales/customer/{clienteId}` y `/sales/summary`.
- **DTOs:** Se constató el correcto mapeo de objetos de solicitud y respuesta (`LoginRequest`, `PaymentRequest`, `SalesRepResponse`, `WarrantyValidationResponse`, `CustomerSalesResponse`, `SalesSummaryResponse` y `ErrorResponse`).
- **Códigos HTTP:** Se comprobó la asociación de códigos de estado de acuerdo a la semántica REST (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found y 500 Internal Server Error).
- **Seguridad:** Se validó la definición global del esquema `BearerAuth` para la protección de endpoints privados.

Vínculo a Evidencia 2 – Validación Swagger:

docs/evidence/swagger_validacion.png

11.2 Validación de la arquitectura

Se verificó la correspondencia y trazabilidad entre las distintas capas del modelo C4 y la API implementada:

- **C4 Nivel 1 (Contexto) → C4 Nivel 2 (Contenedores):** El contenedor *API Backend* absorbe las responsabilidades del *Sistema de Gestión de Ventas* delimitado en el Nivel 1, interactuando con la *Aplicación Web*, la *Pasarela de Pagos*, la *Base de Datos MySQL* y la *Caché de Redis*.
- **C4 Nivel 2 (Contenedores) → C4 Nivel 3 (Componentes):** El contenedor *API Backend* desglosa internamente la lógica en componentes modulares: *Controladores*, *Middleware JWT*, *DTOs*, *Servicios* y *Repositorios*.
- **C4 Nivel 3 (Componentes) → API:** Cada endpoint definido en el contrato OpenAPI se mapea de forma unívoca a un Controlador y un Servicio. Por ejemplo, el endpoint `GET /sales/rep/{vendedorId}` es receptado por el Controlador de Ventas, validado por el Middleware JWT, orquestado por el Servicio de Ventas y consultado vía el Repositorio de Ventas.

11.3 Validación de persistencia

Se demostró la consistencia en la cadena de transformación de datos `API → DTO → Modelo de Datos → Base de Datos (MySQL)`:

1. **Entrada:** El cliente consume `POST /payments/process` enviando un payload alineado al DTO `PaymentRequest (amount, currency, cardToken)`.
2. **Procesamiento:** El Servicio de Ventas procesa el cobro y construye las entidades `Venta` y `Pago`.
3. **Persistencia:** La capa de persistencia ejecuta la transacción sobre las tablas `ventas` y `pagos` en MySQL, respetando los tipos de datos (`DECIMAL(12,2)` para montos, `DATETIME` para fechas) y las claves foráneas (`FOREIGN KEY (venta_id) REFERENCES ventas(venta_id)`).
4. **Salida:** La BD retorna los identificadores generados y la API responde con el DTO `PaymentResponse (transactionId, status, timestamp)`.

11.4 Validación de caché

Se comprobó que la estrategia de caché basada en Redis bajo el patrón Cache-Aside es coherente con las operaciones de lectura y modificación:

- **Operaciones de Lectura (`GET /sales/summary`):** La API consulta primero en Redis. Ante un *Cache Hit*, la respuesta se sirve en milisegundos sin sobrecargar a MySQL. Ante

un *Cache Miss*, la API consulta MySQL, actualiza la clave en Redis con un tiempo de vida de 3600 segundos (TTL) y retorna el resultado.

- **Operaciones de Modificación (POST /payments/process):** Al registrarse una nueva venta en MySQL, la API ejecuta inmediatamente una invalidación activa invocando el comando `DEL` sobre las claves de caché afectadas (ej. `sales:summary:{YYYY-MM}` y `sales:customer:{clienteId}`). Esto fuerza a que la siguiente lectura obtenga datos frescos de la base de datos, garantizando consistencia eventual.

12. CONCLUSIONES

1. **Problemática del sistema legado:** El sistema legado presentaba un elevado acoplamiento monolítico, acoplamiento directo a nivel de base de datos entre módulos (Ventas, Recursos Humanos y Soporte) y ausencia de un contrato de interfaz estandarizado, lo que generaba lentitud en consultas analíticas y riesgos de inconsistencia de datos financieros.
2. **Superioridad de la arquitectura propuesta:** El rediseño mediante una arquitectura en capas, documentada con OpenAPI 3.0.3 y modelada bajo el patrón C4, desacopla las responsabilidades operativas. La API RESTful actúa como una fachada segura que expone DTOs bien definidos, independizando la lógica interna de los consumidores externos.
3. **Atributos de calidad mejorados:**
 - **Rendimiento y Latencia:** Reducción drástica del tiempo de respuesta en consultas recurrentes mediante la caché Redis.
 - **Mantenibilidad:** Separación clara de responsabilidades entre controladores, servicios y repositorios.
 - **Integridad de Datos:** Asegurada a nivel de almacenamiento mediante el motor InnoDB de MySQL con soporte transaccional ACID y restricciones referenciales.
 - **Seguridad:** Incorporación estandarizada de tokens JWT y control de acceso basado en roles.
4. **Riesgos permanentes:** Permanece la dependencia operativa de servicios de terceros (Pasarela de Pagos externa), la cual puede ser un punto único de falla si experimenta caídas de red, así como el riesgo de latencia transitoria si la invalidación de claves en Redis falla durante picos extremos de concurrencia.
5. **Consideraciones previas a producción:** Antes del despliegue en ambiente de producción se debe:
 - Configurar variables de entorno seguras para la firma de secretos JWT y credenciales de base de datos.
 - Establecer políticas de respaldos (*backups*) automatizados e incrementales en MySQL.
 - Implementar monitoreo de APM (*Application Performance Monitoring*) para medir la tasa de *Cache Hit/Miss* en Redis y los tiempos de respuesta de los endpoints de la API.

13. REFERENCIAS

- ❖ Brown, S. (2018). *The C4 model for visualising software architecture*. Structurizr. <https://c4model.com/>
- ❖ Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* (Doctoral dissertation, University of California, Irvine).
- ❖ MySQL. (2026). *MySQL 8.0 Reference Manual: The InnoDB Storage Engine*. Oracle Corporation. <https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html>
- ❖ OpenAPI Initiative. (2021). *OpenAPI Specification v3.0.3*. Linux Foundation. <https://spec.openapis.org/oas/v3.0.3>
- ❖ Redis. (2026). *Redis Documentation: Memory optimization and caching patterns*. Redis Ltd. <https://redis.io/docs/>
- ❖ Richardson, L., & Ruby, S. (2007). *RESTful Web Services*. O'Reilly Media.

ANEXOS

Anexo A. Estructura de archivos del repositorio Git El proyecto se encuentra organizado bajo la siguiente estructura estandarizada de directorios dentro de la rama principal (main):

